

JAVA

Networking com Channels, Reactive Programming e Virtual Threads

Sockets, NIO, HttpClient assíncrono e WebSocket: a comunicação em tempo real da BolsaAgora, na prática

Apostila completa sobre comunicação em rede em Java: fundamentos de TCP e UDP, sockets bloqueantes e não bloqueantes com o NIO, identificação de recursos com URI e URL, requisições HTTP síncronas e assíncronas com HttpClient e CompletableFuture, e comunicação em tempo real com WebSocket — com exemplos aplicados a uma plataforma fictícia de cotações financeiras, a BolsaAgora.

CONTEÚDO DA APOSTILA

- | | | | |
|----|--|----|---|
| 01 | O Papel da Rede em uma Aplicação Java | 13 | Construindo um Channel Manager com Polling |
| 02 | Terminologia de Redes: Hosts, Portas e Endereços IP | 14 | UDP na Prática: DatagramSocket |
| 03 | TCP e UDP: Duas Filosofias de Transporte | 15 | UDP na Prática: DatagramChannel |
| 04 | Sockets: A Porta de Entrada para a Comunicação em Rede | 16 | Comparando TCP e UDP em Baixo Nível |
| 05 | Criando uma Conexão Cliente-Servidor com ServerSocket | 17 | HTTP e o HttpURLConnection |
| 06 | URI, URL e URN: Identificando Recursos na Web | 18 | HttpClient: A Alternativa Moderna |
| 07 | Trabalhando com URI e URL em Java | 19 | Construindo e Enviando Requisições com HttpClient |
| 08 | Visão Geral de Channels e Buffers | 20 | Requisições Assíncronas: send vs. sendAsync e CompletableFuture |
| 09 | Propriedades de um Buffer: Capacity, Limit e Position | 21 | Encadeando Reações: thenAccept, thenApply e thenRun |
| 10 | ServerSocketChannel vs. ServerSocket | 22 | Introdução ao WebSocket |
| 11 | Polling vs. Arquitetura Orientada a Eventos | 23 | Construindo um Chat com WebSocket.Listener |
| 12 | Selectors e Canais Seleccionáveis | | |

Sumário

01 O Papel da Rede em uma Aplicação Java

Por que a BolsaAgora, e qualquer aplicação distribuída, precisa conversar pela rede

02 Terminologia de Redes: Hosts, Portas e Endereços IP

O vocabulário básico para entender qualquer conexão de rede

03 TCP e UDP: Duas Filosofias de Transporte

Confiabilidade contra velocidade, e quando cada uma vence

04 Sockets: A Porta de Entrada para a Comunicação em Rede

Como Socket e ServerSocket abrem e mantêm uma conexão TCP

05 Criando uma Conexão Cliente-Servidor com ServerSocket

Construindo o primeiro servidor de cotações da BolsaAgora

06 URI, URL e URN: Identificando Recursos na Web

Três siglas parecidas, com papéis bem diferentes

07 Trabalhando com URI e URL em Java

Construindo, validando e convertendo endereços de recursos

08 Visão Geral de Channels e Buffers

A base do NIO: pathways de dados e contêineres temporários

09 Propriedades de um Buffer: Capacity, Limit e Position

Os três números que controlam todo Buffer

10 ServerSocketChannel vs. ServerSocket

Trocando bloqueio por escalabilidade

11 Polling vs. Arquitetura Orientada a Eventos

Duas formas de descobrir que algo mudou

12 Selectors e Canais Seleccionáveis

Uma thread só, monitorando centenas de conexões

13 Construindo um Channel Manager com Polling

Um gerenciador de canais próprio para a BolsaAgora

14 UDP na Prática: DatagramSocket

Enviando cotações sem se importar com confirmação

15 UDP na Prática: DatagramChannel

Datagramas com a API de canais do NIO

16 Comparando TCP e UDP em Baixo Nível

Classes, métodos e eventos, lado a lado

17 HTTP e o HttpURLConnection

O protocolo da web, e a primeira forma de falar HTTP em Java

18 HttpClient: A Alternativa Moderna

Por que o Java 11 trouxe um cliente HTTP novo

19 Construindo e Enviando Requisições com HttpClient

A API fluente na prática, buscando cotações

20 Requisições Assíncronas: send vs. sendAsync e CompletableFuture

Não bloquear a thread enquanto a resposta não chega

21 Encadeando Reações: thenAccept, thenApply e thenRun

Callbacks em vez de espera ativa

22 Introdução ao WebSocket

Comunicação bidirecional e persistente entre cliente e servidor

23 Construindo um Chat com WebSocket.Listener

Frames, ping/pong e o chat de operadores da BolsaAgora

O Papel da Rede em uma Aplicação Java

Por que a BolsaAgora, e qualquer aplicação distribuída, precisa conversar pela rede

Uma aplicação que roda sozinha, isolada em uma única JVM, resolve poucos problemas do mundo real. Praticamente toda aplicação relevante hoje precisa trocar dados com outros processos — um banco de dados em outro servidor, uma API de terceiros, um navegador do outro lado do planeta, ou milhares de clientes conectados ao mesmo tempo. Essa troca de dados entre processos que rodam em máquinas diferentes, conectadas por uma infraestrutura física e lógica compartilhada, é o que chamamos de comunicação em rede.

Ao longo desta apostila, vamos construir gradualmente os bastidores de uma plataforma fictícia chamada BolsaAgora. Ela distribui cotações de ativos financeiros em tempo real para milhares de clientes, recebe ordens de compra e venda, e mantém um canal de chat entre operadores. É um cenário rico o bastante para exigir praticamente todo canal de comunicação que o Java oferece: sockets tradicionais, canais não bloqueantes do NIO, requisições HTTP síncronas e assíncronas, e conexões persistentes via WebSocket.

As camadas que uma aplicação Java pode tocar

O Java expõe a comunicação em rede através de pacotes com níveis de abstração bem diferentes entre si. Nos níveis mais baixos, o pacote `java.net` oferece `Socket` e `ServerSocket`, um modelo bloqueante e relativamente simples de entender. Um nível abaixo desse, o pacote `java.nio.channels` expõe `Channels` e `Buffers`, uma API não bloqueante, pensada para aplicações que precisam lidar com muitas conexões simultâneas sem gastar uma thread por cliente. E em um nível mais alto, o pacote `java.net.http` concentra o `HttpClient` — voltado especificamente à comunicação via protocolo HTTP, incluindo suporte nativo a WebSocket.

Pacote	Nível de abstração	Uso típico
<code>java.net</code>	Baixo — sockets bloqueantes	Conexões simples, cliente-servidor tradicional
<code>java.nio / java.nio.channels</code>	Médio — canais não bloqueantes	Servidores escaláveis com muitas conexões
<code>java.net.http</code>	Alto — cliente HTTP e WebSocket	Consumir APIs REST e canais em tempo real

DICA — Por que a BolsaAgora nesta apostila

Um sistema de cotações em tempo real precisa, ao mesmo tempo, ser rápido para muitos clientes simultâneos, conversar com serviços HTTP externos, e manter conexões abertas para atualizações contínuas — por isso ela aparece em praticamente todos os capítulos desta apostila, sob um ângulo diferente a cada vez.

O que este material cobre

- » Os fundamentos de terminologia de rede: hosts, portas, endereços IP, TCP e UDP.
- » Como abrir e manter conexões com Socket, ServerSocket e o modelo bloqueante clássico.
- » Como o NIO reorganiza a comunicação em torno de Channels, Buffers e Selectors.
- » Como identificar e construir recursos de rede com URI e URL.
- » Como consumir serviços HTTP de forma síncrona e assíncrona com HttpClient e CompletableFuture.
- » Como estabelecer uma conexão bidirecional e persistente com WebSocket.

CAPÍTULO 02

Terminologia de Redes: Hosts, Portas e Endereços IP

O vocabulário básico para entender qualquer conexão de rede

Antes de escrever qualquer linha de código de rede, vale alinhar um vocabulário que vai aparecer o resto da apostila inteira. Uma rede, no sentido mais simples, é um conjunto de dispositivos conectados entre si, capazes de trocar dados. Uma rede local, restrita a um prédio ou escritório, costuma ser chamada de intranet; a internet é a rede pública que conecta redes menores entre si, em escala global.

Cada dispositivo participante de uma rede é chamado de host — pode ser um notebook, um celular, um servidor na nuvem, ou qualquer outra máquina capaz de enviar e receber dados. Na comunicação da BolsaAgora, o servidor que centraliza as cotações é um host, e cada aplicativo de um investidor conectado a ele é outro host. Quando dois hosts trocam dados, um papel comum é o de cliente — quem inicia a conversa, pedindo algo — e o outro é o de servidor — quem fica esperando por pedidos, e responde a eles. É esse arranjo, repetido em praticamente toda aplicação de rede, que chamamos de modelo cliente-servidor.

Endereços IP: identificando um host de forma única

Para que um pacote de dados saiba para onde ir, todo host em uma rede TCP/IP recebe um endereço IP — um identificador numérico único dentro daquela rede. A versão mais usada historicamente é a IPv4, escrita como quatro grupos de números separados por pontos, como 192.168.0.42. Só que o IPv4 tem um teto de aproximadamente 4,3 bilhões de endereços possíveis — um número que a internet moderna, com bilhões de dispositivos conectados, já consegue esgotar. É para resolver esse problema que existe o IPv6, com um espaço de endereços bem maior, escrito em grupos hexadecimais separados por dois pontos.

Versão	Formato	Exemplo
IPv4	4 grupos decimais (0–255), separados por ponto	192.168.0.42
IPv6	8 grupos hexadecimais, separados por dois-pontos	fe80::1c2f:8a3d:9e21:44ab

DICA — As duas versões convivem

IPv4 e IPv6 continuam coexistindo hoje: roteadores mais antigos, ou provedores que só distribuem endereços IPv4, mantêm o protocolo mais antigo relevante — por isso uma aplicação de rede robusta deveria ser capaz de lidar com os dois formatos.

Portas: identificando o serviço dentro do host

Um endereço IP localiza o host, mas um mesmo host normalmente roda vários serviços de rede ao mesmo tempo — um servidor web, um banco de dados, uma aplicação de cotações. É para diferenciar esses serviços que existe a porta: um número entre 0 e 65535, associado à combinação de um endereço IP com um protocolo de transporte. A dupla endereço IP mais porta é o que forma, na prática, o endereço completo de um socket.

Algumas portas têm papéis convencionados: a porta 80 costuma ser reservada para tráfego HTTP não criptografado, e a 443 para HTTPS. Portas abaixo de 1024 são chamadas de well-known ports, e normalmente exigem privilégios elevados para que uma aplicação possa escutar nelas. Por isso, ao longo desta apostila, o servidor da BolsaAgora vai escutar em portas mais altas, como 7070 — uma escolha arbitrária seguindo essa convenção comum de usar portas fora da faixa reservada durante o desenvolvimento.

REGRA — Porta é local, não é a mesma coisa que endereço

Uma porta só faz sentido combinada com um endereço IP e um host específico — a porta 7070 do servidor da BolsaAgora não tem nenhuma relação com a porta 7070 de qualquer outra máquina na rede.

CAPÍTULO 03

TCP e UDP: Duas Filosofias de Transporte

Confiabilidade contra velocidade, e quando cada uma vence

Depois que um pacote de dados sabe para qual endereço IP e porta ele precisa viajar, ainda resta decidir como ele vai viajar. Essa é a responsabilidade da camada de transporte, e o Java dá acesso direto aos dois protocolos mais usados nela: o TCP (Transmission Control Protocol), de 1974, e o UDP (User Datagram Protocol), de 1980. Os dois resolvem o mesmo problema — mover bytes de um host a outro — mas partem de prioridades opostas.

O TCP é orientado a conexão: antes de qualquer dado real trafegar, cliente e servidor executam um three-way handshake, uma pequena troca de mensagens que sincroniza os dois lados e confirma que ambos estão prontos para conversar. A partir daí, o TCP garante que os dados cheguem — reenviando pacotes perdidos — e que cheguem na ordem em que foram enviados. Esse cuidado tem um custo: mais overhead de rede, e uma velocidade consideravelmente menor que a do UDP.

Já o UDP é connectionless — não existe handshake, nem confirmação de entrega, nem garantia de ordem. Um datagrama UDP é uma mensagem autocontida, enviada sem que o remetente saiba, ou se importe, se ela chegou. Essa simplicidade tem uma contrapartida direta em velocidade: sem o overhead de controle do TCP, o UDP transmite dados de forma significativamente mais rápida, ao custo de nenhuma garantia de entrega.

Característica	TCP (1974)	UDP (1980)
Tipo de conexão	Orientado a conexão	Sem conexão (connectionless)
Confiabilidade	Alta — reenvia pacotes perdidos	Nenhuma garantia de entrega
Categoria	Fluxo contínuo de dados (stream)	Datagramas (pacotes discretos)
Verificação de erros	Sim	Não
Overhead	Maior	Menor
Velocidade	Mais lento	Mais rápido
Casos de uso	Navegação web, e-mail, transferência de arquivos	Jogos online, streaming, chamadas de voz/vídeo, feeds em tempo real

Para a BolsaAgora, os dois protocolos convivem em papéis diferentes. O envio de ordens de compra e venda — onde perder um dado é inaceitável — usa TCP: uma ordem duplicada ou perdida pode ter consequências financeiras reais. Já a distribuição contínua de cotações de preço, onde um valor perdido é rapidamente substituído pelo próximo, é uma candidata natural ao UDP: se um pacote de preço se perde, o próximo, que chega décimos de segundo depois, já resolve o problema.

DICA — A escolha depende do que você não pode perder

Uma regra prática: se a aplicação não sobrevive a perder ou reordenar uma única mensagem, use TCP. Se a aplicação se importa mais com a mensagem mais recente do que com garantir que todas cheguem, o UDP tende a ser a escolha certa.

CAPÍTULO 04

Sockets: A Porta de Entrada para a Comunicação em Rede

Como Socket e ServerSocket abrem e mantêm uma conexão TCP

Um socket é a abstração que o sistema operacional oferece para que um programa envie e receba dados pela rede — é o ponto de extremidade concreto de uma conexão, identificado pela combinação de endereço IP e porta vista no capítulo 2. O Java expõe essa abstração através de duas classes centrais no pacote `java.net`: `Socket`, que representa a conexão do lado do cliente, e `ServerSocket`, que representa o processo de escutar por novas conexões do lado do servidor.

O papel do `ServerSocket` é relativamente especializado: ele não troca dados diretamente com ninguém. Sua única responsabilidade é vincular-se a uma porta local e aguardar — através do método `accept` — até que um cliente peça para se conectar. Quando isso acontece, o `accept` devolve um novo objeto `Socket`, já conectado àquele cliente específico, e é através desse `Socket` que a troca de dados de fato acontece, tipicamente lendo e escrevendo em um `InputStream` e um `OutputStream`.

Classe	Papel	Método central
<code>ServerSocket</code>	Escuta uma porta e aceita novas conexões	<code>accept()</code>
<code>Socket</code>	Representa uma conexão já estabelecida com um par remoto	<code>getInputStream()</code> / <code>getOutputStream()</code>

Do lado do cliente, o processo é mais direto: basta instanciar um `Socket` informando o endereço do servidor e a porta desejada. O construtor já tenta abrir a conexão TCP, incluindo o three-way handshake visto no capítulo anterior, e devolve um objeto pronto para leitura e escrita assim que a conexão for aceita do outro lado.

```
// Lado cliente – conectando ao servidor de cotacoes da BolsaAgora
Socket conexao = new Socket("localhost", 7070);

try (BufferedReader entrada = new BufferedReader(
    new InputStreamReader(conexao.getInputStream()));
    PrintWriter saida = new PrintWriter(conexao.getOutputStream(), true)) {

    saida.println("ASSINAR PETR4");
    String cotacao = entrada.readLine();
    System.out.println("Cotacao recebida: " + cotacao);
}
```

ATENÇÃO — `accept()` bloqueia a thread

O método `accept()` do `ServerSocket` é bloqueante: a thread que o chama fica parada até que um cliente se conecte. Em um servidor real, é comum delegar cada conexão aceita a uma nova thread, para que o loop principal continue livre para aceitar o próximo cliente — veremos, a partir do capítulo 8, como o NIO resolve essa limitação de outra forma.

CAPÍTULO 05

Criando uma Conexão Cliente-Servidor com ServerSocket

Construindo o primeiro servidor de cotações da BolsaAgora

Com Socket e ServerSocket já apresentados, vale montar o ciclo de vida completo de uma conexão TCP simples, do lado do servidor. O padrão se repete em praticamente qualquer servidor bloqueante: abrir um ServerSocket em uma porta fixa, entrar em um laço infinito chamando accept, e — para cada cliente aceito — tratar aquela conexão individualmente, sem travar o recebimento de novos clientes.

```
public class ServidorCotacoesBolsaAgora {

    private static final int PORTA = 7070;

    public static void main(String[] args) throws IOException {
        try (ServerSocket servidor = new ServerSocket(PORTA)) {
            System.out.println("BolsaAgora escutando na porta " + PORTA);

            while (true) {
                Socket cliente = servidor.accept();
                System.out.println("Cliente conectado: " +
cliente.getInetAddress());
                new Thread(() -> atenderCliente(cliente)).start();
            }
        }
    }

    private static void atenderCliente(Socket cliente) {
        try (BufferedReader entrada = new BufferedReader(
            new InputStreamReader(cliente.getInputStream()));
            PrintWriter saida = new PrintWriter(cliente.getOutputStream(), true))
        {

            String pedido = entrada.readLine();
            if (pedido != null && pedido.startsWith("ASSINAR")) {
                String ativo = pedido.split(" ")[1];
                saida.println(ativo + " = R$ " + buscarCotacaoAtual(ativo));
            }
        } catch (IOException e) {
            System.err.println("Erro ao atender cliente: " + e.getMessage());
        }
    }
}
```

Repare que a thread principal só tem uma responsabilidade: aceitar conexões e delegar cada uma a uma nova thread, imediatamente voltando ao topo do laço para aceitar a próxima. É esse padrão —

uma thread por cliente — que caracteriza o modelo bloqueante tradicional, e é também sua maior limitação: com centenas ou milhares de clientes simultâneos, o custo de manter uma thread do sistema operacional por conexão passa a pesar bastante na performance geral do servidor.

REGRA — try-with-resources fecha sockets automaticamente

Tanto `ServerSocket` quanto `Socket` implementam `AutoCloseable` — abri-los dentro de um bloco `try-with-resources` garante que a conexão seja encerrada corretamente, mesmo se uma exceção interromper o fluxo no meio do caminho.

Do lado do cliente, o fluxo espelha o que já vimos: abrir o `Socket`, obter os streams de entrada e saída, enviar o pedido de assinatura de um ativo, e ler a resposta assim que ela chegar. Vale notar que `readLine()` também bloqueia — a thread do cliente fica parada aguardando até que uma linha completa de resposta chegue pela rede, ou até que a conexão seja encerrada.

CAPÍTULO 06

URI, URL e URN: Identificando Recursos na Web

Três siglas parecidas, com papéis bem diferentes

Sockets resolvem o transporte bruto de bytes, mas boa parte da comunicação de rede moderna acontece em um nível mais alto, identificando recursos por endereço em vez de abrir conexões manualmente. É aí que entram três siglas frequentemente confundidas: URI, URL e URN.

Uniform Resource Identifier, ou URI, é o termo mais abrangente dos três: qualquer string que identifica um recurso, seja pela sua localização, pelo seu nome, ou por ambos. URI é, portanto, uma categoria — e URL e URN são as duas formas mais comuns de URI, cada uma cobrindo metade do problema de identificação.

Sigla	Nome completo	O que identifica
URI	Uniform Resource Identifier	Qualquer identificador de recurso — o termo guarda-chuva
URL	Uniform Resource Locator	Onde o recurso está, e como acessá-lo (protocolo, host, caminho)
URN	Uniform Resource Name	O nome único do recurso, independente de onde ele está hospedado

Uma URL como `https://api.bolsaagora.com/v1/cotacoes/PETR4` é também uma URI: ela diz exatamente onde o recurso está — o host `api.bolsaagora.com`, através do protocolo `https`, no caminho `/v1/cotacoes/PETR4`. Já uma URN, como `urn:isbn:9788535902778`, identifica um recurso pelo nome — nesse exemplo, um livro pelo seu ISBN — sem dizer nada sobre onde encontrá-lo. Toda URL é uma URI, mas nem toda URI é uma URL.

DICA — Na prática, URL domina o dia a dia

URNs são relativamente raras no desenvolvimento cotidiano — a grande maioria das URIs que um desenvolvedor Java escreve são URLs, apontando para um endereço concreto de rede. Ainda assim, a API de Java modela URI como o tipo mais geral, e é dele que URL deriva boa parte de suas capacidades de validação.

CAPÍTULO 07

Trabalhando com URI e URL em Java

Construindo, validando e convertendo endereços de recursos

O Java modela essas duas ideias em classes distintas, ambas no pacote `java.net`: `URI` e `URL`. A classe `URI` é puramente sintática — ela sabe decompor e validar a estrutura de um identificador, mas não sabe, nem tenta saber, como buscar o recurso que ele aponta. Já a classe `URL` carrega essa capacidade extra: sabe abrir uma conexão até o recurso que representa, através do método `openConnection` e de esquemas de protocolo registrados, como `http` e `https`.

Uma URI completa é dividida em componentes bem definidos, cada um com um papel específico na hora de localizar o recurso. Tomando como exemplo `https://api.bolsaagora.com:443/v1/cotacoes?ativo=PETR4#topo`, é possível identificar cada peça dessa estrutura.

Componente	Valor no exemplo	Papel
scheme	https	O protocolo usado para acessar o recurso
host	api.bolsaagora.com	O servidor que hospeda o recurso
port	443	A porta do serviço (opcional; assume um valor padrão por protocolo)
path	/v1/cotacoes	O caminho até o recurso dentro do host
query	ativo=PETR4	Parâmetros adicionais, na forma chave=valor
fragment	topo	Uma referência a uma seção específica dentro do recurso

```
URI enderecoCotacao = new URI(
    "https://api.bolsaagora.com:443/v1/cotacoes?ativo=PETR4#topo");

System.out.println(enderecoCotacao.getScheme()); // https
System.out.println(enderecoCotacao.getHost()); // api.bolsaagora.com
System.out.println(enderecoCotacao.getPort()); // 443
System.out.println(enderecoCotacao.getPath()); // /v1/cotacoes
System.out.println(enderecoCotacao.getQuery()); // ativo=PETR4
System.out.println(enderecoCotacao.getFragment()); // topo

// Convertendo para URL apenas quando for de fato buscar o recurso
URL url = enderecoCotacao.toURL();
```

URI absoluta e URI relativa

Nem toda URI precisa carregar todos esses componentes. Uma URI absoluta, como a do exemplo acima, é autossuficiente: contém tudo o que é necessário para localizar o recurso, independentemente de qualquer contexto. Já uma URI relativa, como `/v1/cotacoes/PETR4`, depende de uma URI base para ser resolvida — ela só faz sentido combinada com um contexto que já definiu o scheme e o host, como quando um link dentro de uma página HTML aponta para outro caminho do mesmo site sem repetir o domínio inteiro.

A sintaxe de uma URI relativa lembra bastante a de um caminho relativo em um sistema de arquivos. Um caminho que começa com uma barra, ou com uma letra de unidade, é absoluto — a raiz está explícita. Já um caminho sem essa barra inicial é relativo ao diretório de trabalho atual, e pode inclusive usar um único ponto para indicar a localização atual, e dois pontos para subir um nível — `v1/cotacoes/PETR4` e `../ordens` seguem exatamente essa mesma lógica, só que aplicada a um endereço de rede em vez de um sistema de arquivos.

```
URI caminhoRelativo = new URI("v1/cotacoes/PETR4");

URL url = caminhoRelativo.toURL();
// lança IllegalArgumentException: URI is not absolute
```

Chamar `toURL()` diretamente em uma URI relativa lança uma `IllegalArgumentException` — não existe informação suficiente na URI para determinar a localização absoluta do recurso, e um URL, por definição, precisa ser absoluto para que o runtime do Java saiba de fato onde buscar o recurso. Ainda assim, URIs relativas são bastante úteis, desde que combinadas com uma URI base — um endereço que fornece o scheme e o host que faltam, servindo como raiz para todos os caminhos relativos construídos a partir dela.

A vantagem prática de trabalhar dessa forma aparece quando uma aplicação acessa muitos endpoints diferentes do mesmo servidor. Em vez de escrever a URI absoluta de cada recurso, repetindo o host da BolsaAgora em cada uma, basta manter uma única URI base e resolver cada caminho relativo contra ela, através do método `resolve`. Se o host da API mudar no futuro — de `api.bolsaagora.com` para `api.bolsaagora.io`, por exemplo — apenas essa base precisa ser

atualizada, em vez de cada ocorrência espalhada pelo código.

```
URI baseBolsaAgora = URI.create("https://api.bolsaagora.com");

URI caminhoCotacao = URI.create("v1/cotacoes/PETR4");
URI caminhoOrdens = URI.create("v1/ordens");

URI enderecoCompletoCotacao = baseBolsaAgora.resolve(caminhoCotacao);
URI enderecoCompletoOrdens = baseBolsaAgora.resolve(caminhoOrdens);

URL urlCotacao = enderecoCompletoCotacao.toURL();
// https://api.bolsaagora.com/v1/cotacoes/PETR4
```

REGRA — Prefira construir URI e converter para URL só no fim

Como a classe URI valida a sintaxe do identificador de forma mais rigorosa, e não tenta abrir nenhuma conexão de rede, é uma prática comum montar e validar o endereço como URI, e só chamar toURL() no último momento, imediatamente antes de efetivamente buscar o recurso.

Construindo uma URL diretamente

É possível também construir uma URL sem passar por uma URI, usando o próprio construtor da classe URL. A diferença de experiência é imediata: enquanto URI.create devolve uma exceção não verificada em caso de erro de sintaxe, o construtor de URL lança MalformedURLException, uma exceção verificada, que precisa ser tratada ou declarada. Existem sobrecargas do construtor que aceitam separadamente o protocolo, o host, a porta e o caminho, mas para o caso mais comum — ler uma página, uma API ou um serviço já com endereço completo — basta informar a string do endereço inteiro.

```
URL urlTeste;
try {
    urlTeste = new URL("https://api.bolsaagora.com/v1/status");
} catch (MalformedURLException e) {
    throw new RuntimeException("Endereco invalido", e);
}
```

Lendo o conteúdo de uma URL com openStream

Uma vez com uma URL válida em mãos, o jeito mais direto de ler o conteúdo do recurso é chamar seu método openStream, que devolve um InputStream pronto para leitura. Nos bastidores, openStream não passa de um atalho: ele encadeia openConnection, que devolve um objeto URLConnection, e getInputStream sobre esse objeto — conveniente quando nenhum ajuste extra de configuração é necessário na conexão.

```

private static void imprimirConteudo(InputStream entrada) {
    try (BufferedReader leitor = new BufferedReader(new
InputStreamReader(entrada))) {
        String linha;
        while ((linha = leitor.readLine()) != null) {
            System.out.println(linha);
        }
    } catch (IOException e) {
        System.err.println("Erro ao ler o recurso: " + e.getMessage());
    }
}

public static void main(String[] args) throws IOException {
    URL urlCotacao = new URL("https://api.bolsaagora.com/v1/cotacoes/PETR4");
    imprimirConteudo(urlCotacao.openStream());
}

```

URLConnection: um processo em duas etapas

Quando é preciso inspecionar ou ajustar a conexão antes de efetivamente ler o recurso — conferir cabeçalhos de resposta, checar o tipo de conteúdo, ou configurar algo antes do pedido seguir — o caminho é chamar `openConnection` manualmente, obtendo um objeto `URLConnection`. É importante entender que abrir essa conexão é um processo de duas etapas: chamar `openConnection` não estabelece, por si só, a conexão de rede real. Esse objeto apenas representa a conexão ainda por vir, e é justamente essa pausa entre criar o objeto e efetivamente conectar que dá espaço para configurá-lo antes do tráfego de rede acontecer — como decidir se a conexão vai suportar leitura, escrita, ou ambas, quando o protocolo permitir.

```

URL urlCotacao = new URL("https://api.bolsaagora.com/v1/cotacoes/PETR4");
URLConnection conexao = urlCotacao.openConnection();

System.out.println(conexao.getContentType()); // application/json; charset=utf-8

conexao.getHeaderFields().entrySet()
    .forEach(System.out::println);

System.out.println(conexao.getHeaderField("cache-control")); // max-age=43200

```

O método `getContentType` devolve o tipo MIME do recurso, e `getHeaderFields` devolve um `Map` com todos os campos de cabeçalho da resposta — informações como tamanho do conteúdo, política de cache, ou o servidor que atendeu o pedido. Quando só um campo específico interessa, `getHeaderField`, recebendo a chave desejada, evita percorrer o mapa inteiro. Um cabeçalho como `cache-control`, por exemplo, informa por quantos segundos uma resposta em cache ainda pode ser considerada válida, sem precisar ser revalidada com o servidor.

A conexão de rede em si só é estabelecida quando o código chama, explicitamente, o método `connect` — ou quando qualquer outro método que dependa da conexão já estar aberta é chamado,

como `getInputStream`, que aciona `connect` implicitamente caso ele ainda não tenha sido chamado antes.

```
URLConnection conexao = urlCotacao.openConnection();
conexao.connect(); // so agora a conexao de rede e de fato estabelecida

imprimirConteudo(conexao.getInputStream());
```

DICA — `openStream` para o caso simples, `openConnection` para o caso configurável

Quando basta ler o conteúdo de um recurso sem nenhum ajuste extra, `url.openStream()` é direto e suficiente. Quando é preciso inspecionar cabeçalhos, checar o tipo de conteúdo antes de decidir como processá-lo, ou configurar a conexão, vale montar o `URLConnection` manualmente através de `openConnection`.

CAPÍTULO 08

Visão Geral de Channels e Buffers

A base do NIO: pathways de dados e contêineres temporários

O modelo de `Socket` e `ServerSocket` visto nos capítulos 4 e 5 é chamado, em conjunto, de IO — o pacote `java.io` original da linguagem. A partir do Java 1.4, a plataforma ganhou um pacote alternativo, o `java.nio` (New I/O, ou Non-blocking I/O), construído em torno de duas abstrações centrais: `Channel` e `Buffer`.

Um `Channel` é uma interface do pacote `java.nio.channels`, que representa uma conexão aberta com alguma entidade capaz de operações de entrada e saída — um arquivo, um dispositivo de hardware, um socket de rede, ou até outro componente de programa. É útil pensar em um `Channel` como o pathway pelo qual os dados trafegam. As implementações mais relevantes para esta apostila são `SocketChannel` e `ServerSocketChannel`, os equivalentes NIO de `Socket` e `ServerSocket`, embora existam outras, como `DatagramChannel` e `FileChannel`.

Um canal é construído chamando o método estático `open` sobre a implementação específica desejada. Nos bastidores, para canais de socket, um socket gerenciado internamente também é aberto — acessível, se necessário, através do método `socket()`. A diferença essencial em relação ao modelo de IO tradicional é como os dados são movidos: em vez de streams contínuos, um `Channel` transfere dados através de `Buffers`, o que dá controle mais fino sobre segmentação de dados e otimização de operações de entrada e saída.

O que é um Buffer

A classe `Buffer` é abstrata, e vive no pacote `java.nio`. Ela representa um contêiner de dados para armazenamento temporário — um bloco de memória que pode ser empurrado para dentro do pathway representado por um `Channel`. Foi desenhada para lidar com dados de forma mais genérica que um simples array: bytes, caracteres, ou outros tipos primitivos, com métodos próprios para acessar, manipular e rastrear o estado interno do buffer.

Um Buffer transita entre quatro estados conceituais ao longo do seu uso: pronto para escrita, pronto para leitura, vazio, e cheio. É justamente esse controle de estado, aliado a um uso de memória mais eficiente que um array simples, que torna o Buffer mais flexível do ponto de vista de performance. Para cada tipo primitivo, exceto boolean, existe um subtipo concreto de Buffer — ByteBuffer, CharBuffer, IntBuffer, e assim por diante.

Como Channel e Buffer interagem

Channels interagem com Buffers para transferir dados nos dois sentidos. O método `read` de um Channel recebe um Buffer como parâmetro, e preenche esse buffer com dados vindos da entidade conectada — o dado então pode ser consumido de dentro do buffer através de seus métodos `get`. No sentido contrário, dados são adicionados a um buffer através dos métodos `put`, e o método `write` de um Channel recebe esse buffer já preenchido, transferindo seu conteúdo para a entidade conectada do outro lado.

```
// Abrindo um canal e lendo dados de um cliente da BolsaAgora
SocketChannel canal = SocketChannel.open();
canal.connect(new InetSocketAddress("localhost", 7070));

ByteBuffer buffer = ByteBuffer.allocate(256);
int bytesLidos = canal.read(buffer); // preenche o buffer com dados do canal

buffer.flip(); // muda o buffer para modo leitura
while (buffer.hasRemaining()) {
    System.out.print((char) buffer.get());
}
```

ATENÇÃO — `java.nio.Buffer` não é o buffer de um Stream com buffer

É fácil confundir o Buffer do NIO com o buffer interno de classes como `BufferedInputStream`. São conceitos diferentes: um `java.nio.Buffer` pode ser lido e escrito pelo mesmo objeto — bastando alternar seu estado com o método `flip` — e foi desenhado para controle fino e transferências em lote, enquanto o buffer de um `InputStream/OutputStream` é uma otimização interna, invisível ao código que o usa.

Aspecto	Buffer de <code>InputStream/OutputStream</code>	<code>java.nio.Buffer</code>
Performance	Mais lento	Mais rápido para transferências em lote
Controle e flexibilidade	Limitado	Controle fino sobre a posição e o conteúdo
Integração	APIs baseadas em stream	I/O de baixo nível, canais NIO, redes

CAPÍTULO 09

Propriedades de um Buffer: Capacity, Limit e Position

Os três números que controlam todo Buffer

Além do seu conteúdo propriamente dito, todo Buffer é regido por três propriedades numéricas essenciais, que juntas determinam o que pode ser lido ou escrito nele a qualquer momento: capacity, limit e position.

Propriedade	Mínimo	Máximo	Mutável?
Capacity	0	Definido na alocação	Não
Limit	0	$\leq$ Capacity	Sim
Position	0	$\leq$ Limit	Sim

A capacity é o número máximo de elementos que o buffer pode armazenar — definida uma única vez, no momento em que o buffer é alocado, e imutável depois disso. O limit marca até onde os dados podem ser lidos ou escritos na operação atual; nunca pode ultrapassar a capacity. Já a position é um cursor, indicando o próximo índice a ser lido ou escrito, e avança automaticamente a cada chamada de get ou put.

O método flip() é o que costuma causar mais confusão em quem está começando com Buffers, mas seu papel é simples: ele prepara um buffer que acabou de ser escrito para ser lido, movendo o limit para a position atual — ou seja, até onde os dados de fato foram escritos — e zerando a position, para que a leitura comece do início. Sem esse passo, tentar ler um buffer logo depois de escrever nele leria posições vazias, além do que foi realmente preenchido.

```
ByteBuffer buffer = ByteBuffer.allocate(128); // capacity = 128, position = 0

buffer.put("PETR4=38,42".getBytes()); // escreve; position avança
System.out.println(buffer.position()); // 11

buffer.flip(); // limit = position (11); position = 0
System.out.println(buffer.limit()); // 11

byte[] dados = new byte[buffer.limit()];
buffer.get(dados); // le ate o limit
System.out.println(new String(dados)); // PETR4=38,42

buffer.clear(); // reseta position=0, limit=capacity
// pronto para uma nova escrita
```

REGRA — flip para ler, clear para reescrever do zero

Um padrão quase universal ao trabalhar com Buffers é: escrever dados, chamar `flip()` para ler o que foi escrito, e — quando o buffer for reutilizado para uma nova escrita do zero — chamar `clear()`, que reseta a `position` para 0 e o `limit` de volta para a `capacity`.

ByteBuffer ou CharBuffer?

Para canais de socket, o `ByteBuffer` costuma ser a escolha certa. Canais de socket operam no nível de bytes, transmitindo dados brutos sem nenhuma codificação de caractere embutida — o `ByteBuffer` se alinha naturalmente a esse modelo, tornando-se a opção mais eficiente para transferência de dados. O `CharBuffer`, embora útil para operações orientadas a caracteres, introduz uma camada extra de codificação e decodificação, que pode impactar a performance quando o gargalo real está na rede.

CAPÍTULO 10

ServerSocketChannel vs. ServerSocket

Trocando bloqueio por escalabilidade

Com Channels e Buffers apresentados, vale comparar diretamente o `ServerSocketChannel`, visto no capítulo anterior, com o `ServerSocket` clássico do capítulo 4 — os dois resolvem o mesmo problema de aceitar conexões, mas com filosofias bem diferentes por trás.

Característica	ServerSocket	ServerSocketChannel
Modelo de I/O	Bloqueante	Não bloqueante (NIO)
Eficiência de threads	Exige uma thread por conexão de cliente	Uma única thread pode atender vários clientes
Escalabilidade	Menos escalável para muitas conexões simultâneas	Altamente escalável para muitas conexões
Tipo de conexão	TCP	TCP, ou qualquer protocolo compatível com NIO

Característica	ServerSocket	ServerSocketChannel
Padrão de transferência	Baseado em stream (fluxo contínuo)	Stream ou baseado em datagramas (pacotes discretos)
Performance	Geralmente mais lento, por sua natureza bloqueante	Potencialmente muito mais rápido, com uso eficiente de recursos
Adequação	Ideal para servidores simples, com tráfego baixo a moderado	Perfeito para servidores de alta performance, com muitas conexões e baixa latência

A vantagem central do `ServerSocketChannel` está em como ele se relaciona com threads. No servidor de cotações do capítulo 5, cada cliente aceito ganhava sua própria thread — uma abordagem que funciona bem até algumas centenas de conexões, mas que começa a sofrer quando a `BolsaAgora` precisa atender dezenas de milhares de investidores conectados ao mesmo tempo, todos aguardando atualizações de preço. Um `ServerSocketChannel`, combinado a um `Selector` — assunto dos próximos capítulos — permite que uma única thread monitore centenas ou milhares de conexões simultaneamente.

ATENÇÃO — A troca por escalabilidade tem um preço

O `ServerSocketChannel` tem uma desvantagem real: sua API é bem mais complexa que a do `ServerSocket`. Programar contra `Buffers`, `Channels` e `Selectors` exige entender detalhes que o modelo bloqueante simplesmente esconde — vale a pena para servidores de alta performance, mas é um exagero para uma aplicação simples com tráfego baixo.

CAPÍTULO 11

Polling vs. Arquitetura Orientada a Eventos

Duas formas de descobrir que algo mudou

Antes de mergulhar nos `Selectors`, vale entender o problema geral que eles resolvem: como um programa descobre que algo aconteceu — um cliente enviou dados, uma conexão foi aceita, um arquivo mudou — sem ficar bloqueado esperando por isso. Existem, de forma ampla, duas estratégias: polling e arquitetura orientada a eventos.

Polling significa checar ativamente por mudanças em intervalos regulares — um laço que pergunta repetidamente 'já aconteceu?', até que a resposta seja sim. É um paradigma mais procedural e familiar, o que o torna mais simples de implementar à primeira vista. O problema é que ele é inerentemente ineficiente e consome recursos mesmo quando nada mudou — cada checagem consome ciclos de CPU, esteja ou não havendo novidade para tratar.

Já uma arquitetura orientada a eventos reage a eventos específicos, em vez de checá-los ativamente. Componentes registram interesse em determinados eventos, e o sistema notifica esses componentes assim que o evento correspondente é disparado — sem que ninguém precise ficar perguntando repetidamente se algo mudou. É um modelo mais escalável e responsivo, e

consideravelmente menos intensivo em recursos, embora normalmente mais difícil de implementar do zero.

	Polling	Orientado a Eventos
Como funciona	Checa ativamente por mudanças em intervalos regulares	Reage a eventos específicos
Desvantagem	Ineficiente e intensivo em recursos	Pode ser mais difícil de implementar
Vantagens	Mais simples de implementar inicialmente, por ser um paradigma mais familiar	Mais escalável e responsivo; menos intensivo em recursos
Casos de uso	Loop de jogo checando entrada do usuário; ping em uma API por novos dados	Clique de botão disparando uma função; fila de mensagens entregando dados

DICA — A BolsaAgora não pode se dar ao luxo do polling ingênuo

Imagine um servidor perguntando, para cada um de dez mil clientes conectados, 'você enviou algo?', dezenas de vezes por segundo. O custo cresceria proporcionalmente ao número de conexões — é exatamente esse cenário que motiva o uso de Selectors, vistos a seguir, que aproximam o NIO de um modelo orientado a eventos.

CAPÍTULO 12

Selectors e Canais Seleccionáveis

Uma thread só, monitorando centenas de conexões

Em uma arquitetura orientada a eventos, eventos esperados são identificados de antemão, e componentes podem registrar interesse em cada um deles. Quando um desses eventos é disparado, o sistema reage, compartilhando a notificação do evento com as partes interessadas. Componentes ficam fracamente acoplados entre si, operando de forma independente, e a comunicação acontece através do disparo e da escuta de eventos — o evento é quem conduz a ação, em vez de um código em loop indefinido tentando identificar mudanças.

No mundo do NIO, os canais capazes de participar desse modelo são chamados de canais seleccionáveis. Um `SelectableChannel` é, sob todos os aspectos, um canal como qualquer outro — representa um ponto de entrada e saída, como um socket, um pipe, ou um canal de arquivo — mas com uma capacidade extra: pode ser associado a um `Selector`, a peça central da programação NIO orientada a eventos. A classe `SelectableChannel` vive no pacote `java.nio.channels`, e tanto `ServerSocketChannel` quanto `SocketChannel` são exemplos concretos desse tipo.

O que um Selector faz

Um `Selector` monitora um conjunto de canais, verificando quais deles estão prontos para um evento específico. Esses eventos às vezes são chamados de interesses, e a aplicação registra o interesse

de um canal em um evento específico através do próprio Selector. Quando o evento acontece em algum dos canais registrados, o Selector acorda o programa, entregando informações sobre o evento e sobre o canal interessado. A partir daí, o programa reage àquele evento específico — tipicamente executando a operação de I/O correspondente, ou tomando qualquer outra ação apropriada.

```
Selector selector = Selector.open();

ServerSocketChannel servidor = ServerSocketChannel.open();
servidor.bind(new InetSocketAddress(7070));
servidor.configureBlocking(false);
servidor.register(selector, SelectionKey.OP_ACCEPT);

while (true) {
    selector.select(); // bloqueia ate que ALGUM canal registrado tenha um evento

    Iterator<SelectionKey> chaves = selector.selectedKeys().iterator();
    while (chaves.hasNext()) {
        SelectionKey chave = chaves.next();

        if (chave.isAcceptable()) {
            SocketChannel novoCliente = servidor.accept();
            novoCliente.configureBlocking(false);
            novoCliente.register(selector, SelectionKey.OP_READ);
        } else if (chave.isReadable()) {
            atenderLeituraDoCliente((SocketChannel) chave.channel());
        }
        chaves.remove();
    }
}
```

Repare que uma única thread executa esse laço inteiro, e ainda assim consegue reagir a eventos de aceitação de novas conexões e de leitura de dados vindos de qualquer um dos clientes já conectados — sem nunca bloquear esperando por um cliente específico. É esse arranjo que permite ao servidor de cotações da BolsaAgora escalar para muitos milhares de conexões simultâneas, usando uma fração das threads que o modelo bloqueante do capítulo 5 exigiria.

REGRA — select() ainda bloqueia — só que de forma coletiva

O método select() do Selector bloqueia a thread, mas de uma forma bem diferente do accept() do ServerSocket tradicional: ele libera a thread assim que qualquer canal registrado tiver um evento pronto, em vez de esperar por um único canal específico. É essa espera coletiva que viabiliza o modelo de uma thread para muitos clientes.

CAPÍTULO 13

Construindo um Channel Manager com Polling

Apesar de Selectors serem a abordagem recomendada para monitorar muitos canais, vale entender também como seria implementar manualmente uma estratégia de polling sobre um conjunto de SocketChannels, tanto para reforçar por que Selectors existem, quanto porque esse padrão aparece em sistemas legados, e em cenários simples o bastante para dispensar a complexidade extra de um Selector.

A ideia central de um Channel Manager por polling é simples: manter uma lista dos canais ativos, e periodicamente perguntar a cada um deles se há dados disponíveis para leitura, usando chamadas não bloqueantes. Diferente do `accept()` ou do `read()` bloqueante vistos antes, um canal configurado como não bloqueante devolve imediatamente, mesmo que não haja nada para ler naquele instante — cabe ao gerenciador decidir o que fazer com essa ausência de dados.

```
public class GerenciadorDeCanaisBolsaAgora {

    private final List<SocketChannel> canaisAtivos = new ArrayList<>();

    public void registrar(SocketChannel canal) throws IOException {
        canal.configureBlocking(false);
        canaisAtivos.add(canal);
    }

    public void executarCicloDePolling() throws IOException {
        ByteBuffer buffer = ByteBuffer.allocate(256);

        for (Iterator<SocketChannel> it = canaisAtivos.iterator(); it.hasNext(); )
        {
            SocketChannel canal = it.next();
            buffer.clear();

            int bytesLidos = canal.read(buffer); // nao bloqueia
            if (bytesLidos > 0) {
                buffer.flip();
                processarPedido(canal, buffer);
            } else if (bytesLidos == -1) {
                canal.close();
                it.remove(); // cliente encerrou a conexao
            }
            // bytesLidos == 0 significa "nada disponivel agora"; segue para o
            proximo canal
        }
    }
}
```

Esse gerenciador precisaria ser chamado repetidamente, em um laço externo, checando todos os canais ativos a cada iteração — mesmo aqueles que não têm nenhum dado novo. Com poucos clientes, o custo é irrelevante. Mas conforme a lista de canaisAtivos cresce para milhares de

conexões, esse laço de polling passa a gastar tempo de CPU checando canais ociosos repetidamente, o exato problema de ineficiência descrito no capítulo anterior.

DICA — Selector é, na prática, um polling delegado ao sistema operacional

A diferença entre esse gerenciador manual e um Selector não é tão grande conceitualmente quanto parece: por trás dos panos, o Selector também depende de mecanismos do sistema operacional para monitorar múltiplos descritores de arquivo. A vantagem é que essa checagem é feita de forma otimizada pelo próprio SO, em vez de um laço Java verificando canal por canal a cada iteração.

CAPÍTULO 14

UDP na Prática: DatagramSocket

Enviando cotações sem se importar com confirmação

Voltando ao UDP apresentado no capítulo 3, é hora de ver sua API concreta em Java. Assim como o TCP tem Socket e ServerSocket, o UDP tem sua própria classe central: DatagramSocket, usada tanto do lado de quem envia quanto do lado de quem recebe. Não há uma distinção entre uma classe de cliente e uma de servidor, como acontecia com Socket e ServerSocket — reflexo direto da natureza connectionless do protocolo.

UDP transporta dados na forma de datagramas — mensagens autocontidas, cada uma carregando tudo o que é necessário para chegar ao destino, sem depender de nenhum estado de conexão prévio. Em Java, um datagrama é representado pela classe DatagramPacket, que empacota tanto os dados quanto o endereço e a porta de destino (ao enviar) ou de origem (ao receber).

Como visto anteriormente, não existe handshake nem fase explícita de estabelecimento de conexão em UDP: um DatagramSocket não precisa negociar nada com o destino antes de enviar dados. Isso torna o protocolo especialmente adequado quando confiabilidade garantida não é essencial, quando não é necessária uma conexão bidirecional persistente, ou quando velocidade é a prioridade — exatamente o perfil da distribuição contínua de preços da BolsaAgora, discutida no capítulo 3.

```
// Servidor UDP — transmite cotacoes para quem perguntar
DatagramSocket servidor = new DatagramSocket(9090);
byte[] bufferRecebe = new byte[256];

while (true) {
    DatagramPacket pedido = new DatagramPacket(bufferRecebe, bufferRecebe.length);
    servidor.receive(pedido); // bloqueia ate chegar um pedido

    String ativo = new String(pedido.getData(), 0, pedido.getLength());
    String cotacao = ativo + "=" + buscarCotacaoAtual(ativo);
    byte[] resposta = cotacao.getBytes();

    DatagramPacket pacoteResposta = new DatagramPacket(
        resposta, resposta.length, pedido.getAddress(), pedido.getPort());
    servidor.send(pacoteResposta);
}
```

```
// Cliente UDP — pergunta o preco de um ativo
DatagramSocket cliente = new DatagramSocket();
byte[] pedido = "PETR4".getBytes();

DatagramPacket pacotePedido = new DatagramPacket(
    pedido, pedido.length, InetAddress.getByName("localhost"), 9090);
cliente.send(pacotePedido);

byte[] bufferResposta = new byte[256];
DatagramPacket resposta = new DatagramPacket(bufferResposta,
    bufferResposta.length);
cliente.receive(resposta);

System.out.println(new String(resposta.getData(), 0, resposta.getLength()));
```

REGRA — Qual protocolo usar depende da aplicação

A escolha entre TCP e UDP não é sobre qual protocolo é 'melhor' de forma absoluta — é sobre qual característica a aplicação em questão não pode abrir mão. Um streaming de áudio compartilhado entre clientes da BolsaAgora, por exemplo, tolera perder um pacote ocasional muito melhor do que tolera atraso — um cenário claramente favorável ao UDP.

CAPÍTULO 15

UDP na Prática: DatagramChannel

Datagramas com a API de canais do NIO

Assim como o TCP ganhou uma versão NIO com `SocketChannel` e `ServerSocketChannel`, o UDP tem seu próprio canal: `DatagramChannel`, também no pacote `java.nio.channels`. Ele une o modelo

de transporte connectionless do UDP com as vantagens do NIO — Buffers, e a possibilidade de operar em modo não bloqueante, inclusive registrado em um Selector junto de canais TCP.

Um `DatagramChannel` recém-criado está aberto, mas não conectado. Diferente de um `SocketChannel`, isso não impede seu uso: um `DatagramChannel` não precisa estar conectado para que os métodos `send` e `receive` funcionem — e é exatamente esse o padrão mais comum de uso, já que ele espelha a natureza connectionless do protocolo.

```
DatagramChannel canal = DatagramChannel.open();
canal.bind(new InetSocketAddress(9090));

ByteBuffer buffer = ByteBuffer.allocate(256);
SocketAddress origem = canal.receive(buffer); // recebe de qualquer remetente

buffer.flip();
String ativo = StandardCharsets.UTF_8.decode(buffer).toString();

ByteBuffer respostaBuffer = ByteBuffer.wrap(
    (ativo + "=" + buscarCotacaoAtual(ativo)).getBytes());
canal.send(respostaBuffer, origem); // responde diretamente ao remetente
```

Ainda assim, é possível conectar um `DatagramChannel` a um endereço remoto específico, invocando seu método `connect`. Isso não estabelece uma conexão persistente no sentido do TCP — o protocolo continua sem handshake — mas fixa o destino padrão do canal, o que evita o overhead das verificações de segurança que, de outra forma, seriam realizadas a cada chamada de `send` e `receive`. Um detalhe importante: um `DatagramChannel` precisa estar conectado para poder usar os métodos `read` e `write`, já que esses métodos, diferente de `send` e `receive`, não aceitam nem devolvem endereços de socket.

Método	Exige conexão prévia?	Uso típico
<code>send(buffer, endereco)</code>	Não	Enviar a um destinatário que pode variar a cada chamada
<code>receive(buffer)</code>	Não	Receber de qualquer remetente
<code>write(buffer)</code>	Sim (<code>connect()</code> antes)	Enviar repetidamente ao mesmo destino, com menos overhead
<code>read(buffer)</code>	Sim (<code>connect()</code> antes)	Ler repetidamente do mesmo remetente conectado

DICA — Conecte quando o destino for fixo

Se um componente da BolsaAgora troca datagramas repetidamente com o mesmo par remoto — como dois servidores internos sincronizando um cache de preços — vale a pena chamar `connect()` uma única vez e usar `read/write` depois, evitando o custo repetido das checagens de segurança do `send/receive`.

Comparando TCP e UDP em Baixo Nível

Classes, métodos e eventos, lado a lado

Com as quatro combinações apresentadas — servidor e cliente, em TCP e em UDP — vale consolidar todas elas em uma única visão comparativa, olhando para as classes envolvidas, os métodos centrais de cada uma, o tipo de dado transportado, e a sequência de eventos típica de cada modelo.

	Servidor TCP	Cliente TCP	Servidor UDP	Cliente UDP
Classe	ServerSocket	Socket	DatagramSocket	DatagramSocket
Métodos centrais	accept	connect	send, receive	send, receive
Dado transportado	InputStream / OutputStream	InputStream / OutputStream	DatagramPacket	DatagramPacket
Sequência de eventos	Bind, Accept, Read/Write	Connect, Read/Write	Bind, Receive, opcionalmente enviar várias vezes	Enviar, opcionalmente receber

O contraste mais revelador dessa tabela está na sequência de eventos. Um servidor TCP segue um roteiro rígido — vincular a porta, aceitar uma conexão, e então ler e escrever repetidamente sobre essa mesma conexão já estabelecida. Um servidor UDP, por outro lado, só precisa vincular a porta e ficar recebendo — cada datagrama que chega é independente do anterior, e o servidor decide, a cada mensagem, se e para onde enviar uma resposta.

Essa diferença estrutural se propaga para as decisões de arquitetura da BolsaAgora: o canal de ordens de compra e venda, que precisa de uma sessão contínua e confiável entre cada investidor e o servidor, é construído sobre TCP — seja com Socket, seja com SocketChannel. Já a transmissão ampla de cotações, onde não existe uma 'conversa' contínua, apenas anúncios de preço que qualquer cliente interessado pode captar, se encaixa naturalmente no modelo UDP.

REGRA — Nem tudo precisa da mesma camada de transporte

Um sistema de rede real raramente usa um único protocolo para tudo. É perfeitamente comum, e recomendável, combinar TCP onde a confiabilidade é inegociável, e UDP onde a velocidade importa mais do que garantir cada mensagem individual.

HTTP e o HttpURLConnection

O protocolo da web, e a primeira forma de falar HTTP em Java

HTTP significa HyperText Transfer Protocol, e é o alicerce da comunicação de dados na World Wide Web. Segue o mesmo modelo cliente-servidor visto nos capítulos anteriores, mas com uma característica importante: HTTP é stateless — cada requisição é tratada de forma independente, sem que o servidor guarde, por conta própria, memória de requisições anteriores daquele mesmo cliente. Uma mensagem HTTP, seja requisição ou resposta, é composta por um header e um corpo opcional. A versão segura do protocolo, que criptografa essa troca, é o HTTPS.

Métodos HTTP

O protocolo suporta diferentes métodos de requisição, às vezes chamados de verbos HTTP. Os mais comuns são GET, POST, PUT e DELETE; métodos adicionais, usados com menos frequência, incluem PATCH, HEAD, OPTIONS, CONNECT e TRACE.

Método	Uso típico na BolsaAgora
GET	Buscar a cotação atual de um ativo
POST	Enviar uma nova ordem de compra ou venda
PUT	Atualizar por completo uma ordem já existente
DELETE	Cancelar uma ordem pendente

Quando um cliente faz uma requisição — o exemplo mais comum sendo o navegador fazendo um GET para carregar uma página — o servidor responde, e essa resposta inclui um código de status. O código 200 significa OK: o servidor encontrou o recurso pedido e conseguiu devolvê-lo. O código 404 indica que o servidor não conseguiu localizar o recurso solicitado. Um código 500 sinaliza que algo deu errado de forma crítica do lado do servidor.

HttpURLConnection: a primeira geração

Historicamente, a forma de fazer requisições HTTP em Java era através da classe HttpURLConnection, no pacote java.net — obtida a partir de uma URL, chamando openConnection(). É uma API funcional, mas verbosa: o desenvolvedor precisa lidar manualmente com a configuração do método HTTP, a leitura do stream de resposta, e boa parte do tratamento de erros.

```
URL url = new URI("https://api.bolsaagora.com/v1/cotacoes/PETR4").toURL();
URLConnection conexao = (URLConnection) url.openConnection();
conexao.setRequestMethod("GET");

int status = conexao.getResponseCode();
if (status == 200) {
    try (BufferedReader leitor = new BufferedReader(
        new InputStreamReader(conexao.getInputStream()))) {
        String linha;
        StringBuilder corpo = new StringBuilder();
        while ((linha = leitor.readLine()) != null) {
            corpo.append(linha);
        }
        System.out.println(corpo);
    }
} else {
    System.out.println("Falha na requisicao: " + status);
}
```

DICA — Documentação oficial

A documentação completa de `URLConnection` está disponível em [docs.oracle.com](https://docs.oracle.com/javase/10/docs/api/java/net/URLConnection.html), no pacote `java.net` da API do JDK — vale a leitura para conhecer cabeçalhos e configurações menos comuns que essa apostila não cobre em detalhe.

CAPÍTULO 18

HttpClient: A Alternativa Moderna

Por que o Java 11 trouxe um cliente HTTP novo

Por muito tempo, boa parte dos desenvolvedores Java considerou as classes de cliente HTTP do pacote `java.net` pouco robustas para uso sério, preferindo alternativas de terceiros, como o `HttpClient` da Apache, o `OkHttp`, ou até a API de cliente do Jetty. Uma versão incubadora de um cliente HTTP aprimorado foi lançada no JDK 9, mas não teve boa recepção. Foi só no JDK 11 que o Java lançou oficialmente o pacote `java.net.http`, com sua própria classe `HttpClient`.

HTTP/1.1 e HTTP/2

Um dos diferenciais mais relevantes do `HttpClient` é seu suporte ao protocolo HTTP/2, introduzido em 2015, com várias vantagens sobre o HTTP/1.1 anterior. O HTTP/2 é mais rápido, usando diversas técnicas para transferir dados de forma mais eficiente, resultando em tempos de carregamento menores. Também é mais eficiente na priorização de conteúdo: o que é mais importante pode ser carregado primeiro, resultando em uma experiência mais fluida. O `URLConnection` do capítulo anterior funciona apenas com HTTP/1.1; o `HttpClient` suporta as duas versões.

Aspecto	HttpClient	HttpURLConnection
Pacote	java.net.http (Java 11+)	java.net (Java 1.1+)
Suporte a protocolo HTTP	HTTP/1.1 e HTTP/2	Apenas HTTP/1.1
Modelo de execução	Síncrono e assíncrono (não bloqueante)	Apenas síncrono (bloqueante)
Recursos embutidos	Cookies, redirecionamentos e processamento de corpo prontos	Exige tratamento manual de cookies, redirecionamentos e corpo
Recursos avançados	Compressão, esquemas de autenticação	Limitados

Aspecto	HttpClient	HttpURLConnection
API	Fluente, baseada em builder	Tradicional, baseada em métodos avulsos
Tratamento da resposta	Streams reativos; funções auxiliares para tipos comuns de resposta	Exige leitura tradicional via stream ou array de bytes
Facilidade de uso	API simplificada, geralmente menos código para tarefas comuns	Verboso para tarefas comuns

Panorama geral

O HttpClient oferece uma alternativa mais moderna ao HttpURLConnection: uma API eficiente e rica em recursos para interações HTTP. A própria Oracle vem priorizando esforços de desenvolvimento no HttpClient, sinalizando uma preferência clara por seu uso em projetos novos. Frameworks e bibliotecas relevantes do ecossistema Java, como o Spring, também vêm adotando cada vez mais o HttpClient do próprio JDK.

O HttpURLConnection ainda é utilizável, mas seu futuro é incerto. Para projetos novos, a recomendação é clara: usar o HttpClient, pela combinação de design moderno, performance e facilidade de uso. Migrar código já existente para o HttpClient também vale a pena, tanto pela manutenibilidade quanto pelo alinhamento com os rumos futuros do desenvolvimento Java.

REGRA — HttpClient é o padrão para código novo

Salvo alguma restrição concreta de compatibilidade com um projeto legado, toda nova integração HTTP da BolsaAgora, a partir daqui, vai usar HttpClient — inclusive as requisições assíncronas dos próximos capítulos.

CAPÍTULO 19

Construindo e Enviando Requisições com HttpClient

A API fluente na prática, buscando cotações

Diferente do `HttpURLConnection`, obtido a partir de uma URL, o `HttpClient` é construído através de um builder próprio, `HttpClient.newBuilder()`, o que permite configurar aspectos como versão do protocolo, política de redirecionamento e timeout antes mesmo de montar qualquer requisição. Uma vez construído, o mesmo `HttpClient` pode — e deve — ser reaproveitado para várias requisições ao longo da aplicação, já que ele mantém internamente um pool de conexões.

```
HttpClient client = HttpClient.newBuilder()
    .version(HttpClient.Version.HTTP_2)
    .connectTimeout(Duration.ofSeconds(5))
    .build();
```

Uma requisição, por sua vez, é montada através de `HttpRequest.newBuilder()`, informando a URI de destino e, quando necessário, o método HTTP, cabeçalhos e corpo. Por padrão, uma requisição sem método explícito assume GET.

```
HttpRequest requisicao = HttpRequest.newBuilder()
    .uri(URI.create("https://api.bolsaagora.com/v1/cotacoes/PETR4"))
    .header("Accept", "application/json")
    .GET()
    .build();

HttpResponse<String> resposta = client.send(
    requisicao, HttpResponse.BodyHandlers.ofString());

System.out.println("Status: " + resposta.statusCode());
System.out.println("Corpo: " + resposta.body());
```

O segundo argumento de `send` é um `BodyHandler`, responsável por decidir como o corpo da resposta deve ser processado e em que tipo Java ele deve ser entregue. `BodyHandlers.ofString()` devolve o corpo como uma `String` — uma escolha comum para respostas JSON simples, como as cotações da BolsaAgora — mas a mesma API oferece variantes para arquivos, bytes brutos, e streams, entre outras.

Enviando uma ordem com POST

Enviar uma ordem de compra segue a mesma estrutura de builder, mudando o método para POST e informando o corpo da requisição através de um `BodyPublisher`.

```
String corpoJson = """
    {"ativo": "PETR4", "quantidade": 100, "tipo": "COMPRA"}
    """;

HttpRequest ordem = HttpRequest.newBuilder()
    .uri(URI.create("https://api.bolsaagora.com/v1/ordens"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(corpoJson))
    .build();

HttpResponse<String> resposta = client.send(
    ordem, HttpResponse.BodyHandlers.ofString());

if (resposta.statusCode() == 201) {
    System.out.println("Ordem registrada: " + resposta.body());
}
```

DICA — Um HttpClient, muitas requisições

Criar um novo HttpClient para cada requisição desperdiça o pool de conexões que ele mantém internamente. O padrão recomendado é instanciar um único HttpClient — geralmente como um campo estático ou injetado — e reutilizá-lo em toda a aplicação.

CAPÍTULO 20

Requisições Assíncronas: send vs. sendAsync e CompletableFuture

Não bloquear a thread enquanto a resposta não chega

O método `send`, usado até aqui, é síncrono: a thread que o chama fica bloqueada até que a resposta completa do servidor chegue. Para uma aplicação que dispare poucas requisições, isso raramente é um problema — mas a BolsaAgora, ao atualizar simultaneamente as cotações de dezenas de ativos diferentes, não pode se dar ao luxo de esperar cada requisição terminar antes de disparar a próxima. É para isso que existe o método `sendAsync`.

	Tipo de retorno	Invocação do método	Parâmetros do método
Síncrono	<code>HttpResponse<T></code>	<code>send</code>	<code>HttpRequest</code> , <code>BodyHandler<T></code>
Assíncrono	<code>CompletableFuture<HttpResponse<T>></code>	<code>sendAsync</code>	<code>HttpRequest</code> , <code>BodyHandler<T></code>

```
// Envio síncrono
HttpResponse<String> resposta = client.send(
    requisicao, HttpResponse.BodyHandlers.ofString());

// Envio assíncrono
CompletableFuture<HttpResponse<String>> respostaFutura = client.sendAsync(
    requisicao, HttpResponse.BodyHandlers.ofString());
```

A diferença mais visível entre as duas chamadas é o tipo de retorno. Um Future é uma interface que representa o resultado de uma computação assíncrona — um valor que ainda não existe, mas que vai existir em algum momento no futuro. A CompletableFuture é a classe concreta que implementa Future, e estende suas capacidades especificamente para suportar processamento assíncrono.

A própria documentação oficial da API descreve a CompletableFuture como um Future que pode ser explicitamente completado — definindo seu valor e status manualmente — e que também pode ser usado como um CompletionStage, suportando funções e ações dependentes, disparadas quando a computação é concluída. É útil pensar nela como uma promessa de um resultado futuro.

O que é um callback

Um callback é um conceito geral de programação: passar uma função como argumento para outra função, para que ela seja chamada mais tarde, quando um evento específico acontecer. Expressões lambda, quando usadas como argumento de método, não são executadas imediatamente — só quando o método que as recebeu de fato as invoca. Callbacks seguem a mesma lógica: eles não executam até que um determinado estágio, ou evento disparador, ocorra.

Uma vez que uma requisição foi disparada com sendAsync, existem duas formas gerais de reagir à resposta quando ela chegar: registrar um callback — uma abordagem orientada a eventos — ou fazer polling sobre o status de conclusão do CompletableFuture, em um fluxo mais linear e procedural.

```
// Abordagem por callback (orientada a eventos)
respostaFutura.thenAccept(ServicoCotacoes::processarResposta);

while (true) {
    // outras tarefas continuam acontecendo aqui,
    // sem bloquear esperando a resposta
}
```



```
// Abordagem por polling do status (fluxo linear)
while (!respostaFutura.isDone()) {
    // aguardando, possivelmente fazendo outra coisa util
}

HttpResponse<String> resposta = respostaFutura.join();
processarResposta(resposta);
```

REGRA — `sendAsync` nunca bloqueia a thread chamadora

Chamar `sendAsync` devolve o controle imediatamente, com um `CompletableFuture` ainda incompleto — bloquear só acontece se, e quando, o código explicitamente chamar `join()` ou `get()` sobre esse future antes que ele tenha terminado.

CAPÍTULO 21

Encadeando Reações: `thenAccept`, `thenApply` e `thenRun`

Callbacks em vez de espera ativa

O `Selector`, apresentado nos capítulos 12 e 13, é uma ferramenta de baixo nível, desenhada especificamente para operações de I/O não bloqueantes. A `CompletableFuture`, com seu suporte a funções de callback, é uma ferramenta de nível mais alto, e não específica a um único caso de uso — é uma classe de propósito geral, desenhada para gerenciar computações assíncronas de qualquer natureza, não apenas requisições de rede.

A API de `CompletableFuture` oferece uma família de métodos para reagir à conclusão de uma computação, cada um adequado a uma necessidade diferente. `thenAccept` recebe um `Consumer`, usado quando o resultado da requisição precisa ser consumido, mas nada é devolvido em troca. `thenApply` recebe uma `Function`, transformando o resultado em outro valor, que por sua vez pode ser encadeado a novas reações. E `thenRun` recebe um `Runnable`, executado após a conclusão sem que o resultado em si seja necessário.

Método	Tipo funcional aceito	Quando usar
<code>thenAccept</code>	<code>Consumer<T></code>	Consumir o resultado, sem devolver nada
<code>thenApply</code>	<code>Function<T, R></code>	Transformar o resultado em outro valor
<code>thenRun</code>	<code>Runnable</code>	Executar uma ação que não depende do resultado

```
CompletableFuture<HttpResponse<String>> respostaFutura =
    client.sendAsync(requisicaoCotacao, HttpResponse.BodyHandlers.ofString());

respostaFutura
    .thenApply(HttpResponse::body) // extrai so o corpo da resposta
    .thenApply(ServicoCotacoes::converterParaCotacao) // transforma o JSON em
objeto
    .thenAccept(cotacao ->
        painelDeCotacoes.atualizar(cotacao)) // consome o resultado final
    .thenRun(() ->
        System.out.println("Painel atualizado.)); // roda depois, sem precisar do
valor
```

Esse encadeamento é o que torna a `CompletableFuture` particularmente poderosa: cada método devolve uma nova `CompletableFuture`, permitindo compor uma sequência inteira de transformações e reações, todas executadas de forma assíncrona, sem que nenhuma etapa bloqueie a thread principal da aplicação. Para a *BolsaAgora*, isso significa disparar requisições para dezenas de ativos simultaneamente, e deixar que cada resposta atualize o painel de cotações assim que estiver pronta, na ordem em que cada uma efetivamente chegar.

DICA — Callback e Selector resolvem o mesmo problema, em níveis diferentes

Um `Selector` cuida do detalhe de baixo nível de saber quando um socket está pronto para leitura ou escrita. Uma `CompletableFuture` cuida do detalhe de mais alto nível de saber quando um resultado está pronto para ser consumido — não importa se ele veio de uma requisição HTTP, de uma consulta a banco, ou de qualquer outro cálculo assíncrono.

CAPÍTULO 22

Introdução ao WebSocket

Comunicação bidirecional e persistente entre cliente e servidor

Toda a comunicação HTTP vista até aqui compartilha uma limitação estrutural: ela é unidirecional. O cliente faz uma requisição, e o servidor, opcionalmente, responde. Isso coloca sobre o cliente o ônus de se manter atualizado, fazendo polling regular ao servidor para checar por novidades — exatamente o padrão de polling discutido no capítulo 11, com as mesmas desvantagens de ineficiência.

O `WebSocket` resolve esse problema criando uma conexão persistente entre servidor e cliente, que pode ser usada para enviar mensagens nos dois sentidos, a qualquer momento. Isso o torna ideal para aplicações em tempo real, como chats, tickers de ações — o caso exato da *BolsaAgora* — e ferramentas colaborativas, onde os dados precisam ser constantemente atualizados e distribuídos. O `WebSocket` permite que dados sejam empurrados do servidor para o cliente, e vice-versa, sem o overhead de estabelecer uma nova conexão repetidamente.

Do HTTP ao WebSocket: o upgrade

WebSockets aproveitam a confiabilidade das conexões TCP já vistas nos primeiros capítulos. A conexão começa como uma requisição HTTP normal, com um cabeçalho especial de Upgrade, e é isso que caracteriza o chamado handshake HTTP. Esse handshake abre caminho para a mudança para o protocolo WebSocket propriamente dito, com troca de dados em tempo real, baixo overhead e baixa latência. Esse processo de upgrade garante compatibilidade com a infraestrutura web já existente, e ao mesmo tempo viabiliza uma comunicação mais eficiente e responsiva para aplicações interativas.

Comunicar-se via WebSocket envolve mais complexidade do que uma conexão HTTP padrão. WebSockets exigem esse mecanismo especial de handshake ao estabelecer a conexão, usam um formato de mensagem específico, e tipicamente adotam um formato binário de enquadramento — texto precisa ser codificado antes de ser enviado, e decodificado do lado de quem recebe.

DICA — Referência da Mozilla

A documentação da MDN Web Docs, em developer.mozilla.org, na seção de WebSockets API, é uma referência sólida sobre a escrita de um servidor WebSocket em Java, complementar ao que esta apostila cobre do lado do cliente.

A implementação de WebSocket do Java

O pacote `java.net.http` oferece uma classe chamada `WebSocket`. É essencial entender que essa classe é um cliente WebSocket — o JDK não fornece, nativamente, uma implementação de servidor WebSocket; para o lado servidor, é comum recorrer a frameworks como Spring ou Jakarta EE, ou a bibliotecas dedicadas. Ainda assim, o cliente embutido é suficiente para boa parte das integrações, incluindo o painel de operadores da BolsaAgora que vamos construir no próximo capítulo.

CAPÍTULO 23

Construindo um Chat com `WebSocket.Listener`

Frames, ping/pong e o chat de operadores da BolsaAgora

A interface `Listener` é declarada dentro do tipo `java.net.http.WebSocket`, e é descrita pela própria documentação como a interface de recepção de um WebSocket. Mesmo que nenhum de seus métodos seja sobrescrito, os processos internos do `Listener` continuam acontecendo — ele já cuida dos detalhes de abrir a conexão, receber dados por ela, e responder a pings enviados pelo servidor. Para adicionar tratamento extra, o padrão é sobrescrever o método individual desejado, e dentro dele chamar o método correspondente da superclasse, preservando o comportamento padrão.

Frames: a unidade básica de uma mensagem

É comum ouvir o termo `frame` para descrever as mensagens, ou `payloads`, trocados entre clientes e o servidor WebSocket. `Frame` se refere à unidade básica de dado enviada pela conexão — uma unidade estruturada de informação, que pode carregar dados ou sinais de controle. Frames de controle são trocados ao abrir ou fechar conexões, ou ao fazer ping em uma conexão; frames de dados carregam o conteúdo real da mensagem. Quando uma mensagem é grande demais para

caber em um único frame, ela pode ser dividida em vários frames menores, cada um transmitido separadamente, e depois remontados pelo endpoint que os recebe, para reconstruir a mensagem original completa.

Os métodos de controle do Listener

Em todos os métodos do Listener, o primeiro parâmetro é sempre a própria instância do WebSocket que disparou o evento.

Modificador	Tipo de retorno	Método e parâmetros
default	void	onOpen(WebSocket websocket)
default	void	onError(WebSocket websocket, Throwable error)
default	CompletionStage<?>	onClose(WebSocket websocket, int statusCode, String reason)
default	CompletionStage<?>	onPing(WebSocket websocket, ByteBuffer message)
default	CompletionStage<?>	onPong(WebSocket websocket, ByteBuffer message)

O método `onClose` é chamado com um código de status e um motivo pelo qual a conexão foi encerrada. Alguns códigos de status comuns ajudam a diagnosticar por que uma sessão do chat de operadores foi interrompida.

Código	Significado
1000	Encerramento normal
1001	Indo embora (going away)
1006	Encerramento anormal
1008	Violação de política
1009	Mensagem grande demais

Os frames de controle de ping e pong existem para manter a conexão viva, e confirmar que ambos os lados continuam ativos. Quando uma das partes — cliente ou servidor — quer checar se a outra ainda está respondendo, ela envia um frame de ping. Ao receber um ping, o destinatário deve responder com um frame de pong, confirmando que continua ativo. Essa troca ajuda a detectar conexões que pararam de responder, permitindo reconexão a tempo ou tratamento de erro quando necessário — reforçando a confiabilidade e a estabilidade geral da conexão.

Os métodos de dados: `onText` e `onBinary`

Existem dois métodos de frame de dados. `onText` facilita muito o envio e recebimento de mensagens de texto — o formato usado pelas mensagens do chat de operadores da BolsaAgora. Como também é possível haver mensagens com componentes binários, como imagens ou áudio

anexados a uma mensagem, existe `onBinary` para esses casos.

Modificador	Tipo de retorno	Método e parâmetros
default	<code>CompletionStage<?></code>	<code>onText(WebSocket webSocket, CharSequence data, boolean last)</code>
default	<code>CompletionStage<?></code>	<code>onBinary(WebSocket webSocket, ByteBuffer data, boolean last)</code>

```
WebSocket.Listener listenerDoChat = new WebSocket.Listener() {

    @Override
    public void onOpen(WebSocket webSocket) {
        System.out.println("Conectado ao chat de operadores da BolsaAgora.");
        WebSocket.Listener.super.onOpen(webSocket);
    }

    @Override
    public CompletionStage<?> onText(WebSocket webSocket, CharSequence data,
    boolean last) {
        System.out.println("Mensagem recebida: " + data);
        return WebSocket.Listener.super.onText(webSocket, data, last);
    }

    @Override
    public CompletionStage<?> onClose(WebSocket webSocket, int statusCode, String
    reason) {
        System.out.println("Chat encerrado (" + statusCode + "): " + reason);
        return WebSocket.Listener.super.onClose(webSocket, statusCode, reason);
    }
};

WebSocket chat = HttpClient.newHttpClient()
    .newWebSocketBuilder()
    .buildAsync(URI.create("wss://chat.bolsaagora.com/operadores"),
    listenerDoChat)
    .join();

chat.sendText("Ordem grande de PETR4 chegando, atencao no book.", true);
```

REGRA — Sempre chame o método correspondente da superclasse

Ao sobrescrever um método do `Listener`, chamar `WebSocket.Listener.super.metodo(...)` preserva o comportamento padrão da interface — como responder automaticamente a pings — que de outra forma seria perdido. Esquecer essa chamada é uma causa comum de conexões `WebSocket` que parecem travar sem motivo aparente.

Fim de apostila

Este material percorreu a comunicação em rede em Java de ponta a ponta: os fundamentos de terminologia, TCP e UDP, sockets bloqueantes com `Socket` e `ServerSocket`, o modelo não bloqueante do NIO com Channels, Buffers e Selectors, a identificação de recursos com URI e URL, requisições HTTP síncronas e assíncronas com `HttpClient` e `CompletableFuture`, e a comunicação bidirecional e persistente do `WebSocket`.

Esses conceitos sustentam qualquer aplicação Java que precise conversar com outros processos de forma confiável e eficiente — de um sistema de cotações em tempo real, como a *BolsaAgora* usada em exemplos ao longo desta apostila, a arquiteturas distribuídas de maior porte.